

10 erros que mandam dev junior pro CrashLoopBackOff

Os bugs mais comuns de Kubernetes — e como resolver em 5 minutos.

DEVOPSRAIZ

Tiago Alves da Rocha

Recorte do Ebook 2 da Trilha DEVOPSRAIZ

Antes de começar

Esse PDF é um recorte enxuto do Ebook 2 da Trilha DEVOPSRAIZ — que cobre Docker, Kubernetes e Terraform do zero até produção.

Aqui dentro você tem os 10 erros mais frequentes que param o cluster — com diagnóstico, causa raiz e fix copy-paste pra cada um.

Quem sou eu

Sou Tiago, engenheiro de software há 15 anos focado em Cloud, DevOps e IA aplicada. Já operei Kubernetes em produção em ambientes multi-cloud (AWS, Azure, GCP, OCI) e em 2026 lancei a Trilha DEVOPSRAIZ — 6 ebooks levando do zero a produção.

Como usar

Salva esse PDF no celular. Quando aparecer um erro chato no **kubectl describe pod**, abre aqui e busca o nome do erro na tabela. Cada erro vem com 3 itens: **O que é**, **Por que rola** e **Como resolve**.

Dica do Tiago: antes de qualquer fix, sempre roda **kubectl describe pod <nome>** e **kubectl logs <nome> --previous**. 90% dos erros são óbvios depois disso.

Os 10 erros (parte 1/3)

01 CrashLoopBackOff

STATUS: `CrashLoopBackOff (restart count: 7)`

O que é: Container inicia, crasha, K8s reinicia, crasha de novo. Loop infinito.

Por que rola: Aplicação tá lançando exception no startup — geralmente env var faltando, porta já em uso, ou conexão DB negada.

Como resolve: `kubectl logs <pod> --previous` — mostra logs da última tentativa antes do crash. 99% das vezes a stack trace tá ali.

02 ImagePullBackOff

Failed to pull image "meu-app:v1.2.3": rpc error

O que é: K8s não consegue baixar a imagem do registry.

Por que rola: Tag errada, registry privado sem credencial, ou rede do node não alcança o registry.

Como resolve: Confere o nome da tag (`docker pull` manualmente). Se for registry privado, cria `imagePullSecrets` no Service Account ou no spec do pod. ECR? Garante que o IAM Role do node tem permissão `ecr:GetAuthorizationToken`.

03 OOMKilled

Last State: Terminated Reason: OOMKilled Exit Code: 137

O que é: Container estourou o limite de memória, kernel matou o processo.

Por que rola: Você setou `resources.limits.memory: 256Mi` mas a aplicação consome 400Mi sob carga.

Como resolve: Sobe o limit. Mas antes, profile o app com `kubectl top pod` em pico real. Se for Java, ajusta `-Xmx` pra 75% do limit. Se for Node.js, considera `--max-old-space-size`.

Os 10 erros (parte 2/3)

04 Pending (Unschedulable)

```
0/3 nodes are available: insufficient cpu
```

O que é: Pod tá em Pending pra sempre — nenhum node tem recurso pra ele.

Por que rola: Você pediu CPU/memória que nenhum node disponível tem livre. Ou colocou nodeSelector/tolerations que ninguém atende.

Como resolve: `kubectl describe pod` — vai listar exatamente qual restrição não bateu. Soluções: scale up node group, ajusta requests pra valor real (não chuta), ou usa `cluster-autoscaler`.

05 Evicted

```
Status: Failed Reason: Evicted Message: The node was low on resource: memory
```

O que é: K8s tirou o pod do node porque o node ficou apertado.

Por que rola: Node ficou sem memória/disco e o kubelet começou a despejar pods de menor prioridade.

Como resolve: Ajusta `resources.requests` em todos os pods (assim o scheduler respeita capacidade real). Configura `PriorityClass` nos workloads críticos. Considera HPA pra distribuir carga.

06 Init container falhando

```
Init:Error Init Containers: 0/2
```

O que é: Um initContainer (que roda antes do app) quebrou e bloqueou o pod.

Por que rola: Init container tenta migration de DB, espera de service, ou setup de filesystem — e algum desses pré-requisitos não tá pronto.

Como resolve: `kubectl logs <pod> -c <init-container-name>` mostra o que falhou. Init containers herdam recursos do pod — cuidado com timeout.

07

Liveness probe matando o pod

```
Liveness probe failed: HTTP probe failed with statuscode: 500
```

O que é: Pod fica saudável por 30s, depois K8s mata achando que travou.

Por que rola: `livenessProbe` aponta pra um endpoint que demora pra responder, ou que retorna 500 quando o app tá ocupado.

Como resolve: Endpoint de liveness tem que ser **burro e rápido** — só verifica se o processo respira. Coisas como conexão com DB vão em `readinessProbe` (tira do load balancer mas não mata). Aumenta `initialDelaySeconds` pro app ter tempo de subir.

Os 10 erros (parte 3/3)

08 ConfigMap/Secret não montado

```
MountVolume.SetUp failed for volume "app-config": configmap not found
```

O que é: Pod tenta montar ConfigMap ou Secret que não existe naquele namespace.

Por que rola: Namespace errado, nome digitado errado, ou esqueceu de aplicar o YAML antes do deploy.

Como resolve: `kubectl get configmap -n <namespace>` pra confirmar. Se usa Helm/Kustomize, confere se o template tá renderizando o nome certo. Considera External Secrets Operator pra evitar erro humano.

09 RBAC: Permission denied

```
Error from server (Forbidden): pods is forbidden: User cannot list resource
```

O que é: ServiceAccount do pod não tem permissão pra fazer o que tá tentando.

Por que rola: Aplicação chama API do K8s (ex: leader election, descoberta de pods) mas o SA dela não tem Role/ClusterRole binding.

Como resolve: Cria **Role** com permissões mínimas, faz **RoleBinding** pro ServiceAccount do pod. Princípio do menor privilégio: **nunca** usa **cluster-admin** em workload de produção.

10 PVC Pending

```
Status: Pending Reason: WaitForFirstConsumer
```

O que é: PersistentVolumeClaim fica em Pending, pod não sobe.

Por que rola: StorageClass não existe, ou EBS/Disk Provisioner não configurado, ou AZ do node não tem volume disponível.

Como resolve: `kubectl describe pvc <nome>` mostra o motivo exato. Em EKS, garante que o EBS CSI Driver tá instalado e o IAM Role do node tem **ec2:CreateVolume**. Em GKE/AKS, é mais transparente mas confere o `storageClassName`.

Você sobreviveu aos 10 erros — e agora?

Esse PDF foi um **recorte** do Ebook 2 da Trilha DEVOPSRAIZ. Lá dentro você tem o pacote completo:

Ebook 1	Plataforma Multi-Cloud com IA (FinOps em 4 clouds)
Ebook 2	Docker, Kubernetes, Terraform — do zero a EKS/AKS
Ebook 3	De Projeto a SaaS multi-tenant com billing
Ebook 4	IA Avançada: RAG, Agents, Function Calling
Ebook 5	Observabilidade e SRE em produção
Ebook 6	Segurança Cloud, LGPD, Zero Trust

CUPOM FUNDADOR · 20% OFF

Use **FUNDADOR20** no checkout. Válido só pros 100 primeiros que entram na lista.

Trilha completa »

go.hotmart.com/S105313699A

Me segue lá no **@devopsraiz_oficial** — todo dia tem post novo de Cloud, IA e DevOps em PT-BR. Sem fluff, só código real.